1. What is the difference between frontend and backend JavaScript?

Ans. JavaScript is a programming language used to develop interactive web applications. It can run on both the frontend and the backend.

Frontend JavaScript runs in the user's web browser and is responsible for the interactive parts of a website, such as handling button clicks, validating forms, and updating webpage content.

Backend JavaScript runs on a server, commonly using Node.js, and is responsible for processing requests, interacting with databases, handling authentication, and managing application logic.

Frontend JavaScript	**Backend JavaScript**
Runs in the web browser.	Runs on the server.
Manages user interface and interactions.	Manages server-side logic and data.
Can manipulate the DOM.	Typically does not directly manipulate the browser DOM.
Handles client-side form validation.	Handles database operations and server-side validation.
Example: JavaScript in a webpage.	Example: JavaScript using Node.js and Express.

2. What is DOM?

Ans. DOM stands for Document Object Model. It is a programming interface that represents an HTML or XML document as a hierarchical tree of objects. JavaScript uses the DOM to access, modify, add, or remove webpage elements and their content.

Each HTML element is represented as a node in the DOM tree. For example, headings, paragraphs, and buttons are represented by element nodes.

Example :

```
<p id="message">Hello</p>

<script>

  document.getElementById("message").textContent = "Welcome!";

</script>
```

Uses of the DOM:

- Changing HTML content and CSS styles.
- Adding or removing elements.
- Handling user events.
- Creating dynamic and interactive webpages.

The DOM acts as a bridge between HTML documents and JavaScript, allowing webpages to change dynamically without requiring a complete page reload.

3. What is event in JavaScript?

Ans. An event is an action or occurrence detected by the browser that JavaScript can respond to. Events may be triggered by the user or by the browser itself.

Examples of events include clicking a button, pressing a keyboard key, moving the mouse, submitting a form, and loading a webpage.

JavaScript uses event handlers or event listeners to execute code when an event occurs.

Example :

```
<button id="myButton">Click Me</button>
<script>
  document.getElementById("myButton").addEventListener("click", function() {
    alert("Button clicked!");
  });
</script>
```

Explanation:

- getElementById() selects the button.
- addEventListener() registers an event listener.
- "click" specifies the event to detect.
- The function executes when the button is clicked.

Common JavaScript events:

Event	**Description**
click	Occurs when an element is clicked.
mouseover	Occurs when the pointer moves over an element.
keydown	Occurs when a keyboard key is pressed down.
submit	Occurs when a form is submitted.
load	Occurs when a resource or page finishes loading.

4. What is the event loop in JavaScript?

Ans. The event loop is a mechanism that allows JavaScript to handle asynchronous operations while executing code on its main thread. It coordinates the call stack, task queues, and asynchronous callbacks.

JavaScript normally executes one piece of JavaScript code at a time on its main thread. When asynchronous operations are involved, the event loop helps schedule their callbacks for execution when the call stack is available.

Main components:

1. Call Stack: Executes JavaScript functions.
2. Web APIs: Handle browser-provided operations, such as timers and DOM events.
3. Task Queue: Holds callbacks for tasks such as timer callbacks and certain events.
4. Microtask Queue: Holds microtasks, including Promise handlers.
5. Event Loop: Coordinates when queued callbacks can execute.

Conclusion: The event loop enables JavaScript to coordinate asynchronous tasks without blocking the main thread while synchronous code is executing.

6. What is hoisting in JavaScript?

Ans. Hoisting is JavaScript's behavior of processing declarations before executing the code in their scope. This can make certain variables and functions accessible before the line where they appear in the source code, although their behavior depends on the type of declaration.

Hoisting with var :

console.log(number);

var number = 10;

o/p = undefined

Important points:

- var declarations are hoisted and initialized with undefined.
- let and const declarations are hoisted but cannot be accessed before initialization.
- Function declarations can generally be called before their declaration.
- Function expressions and arrow functions follow the hoisting behavior of the variable used to store them.

6. What is event bubbling?

Ans. Event bubbling is a process in which an event triggered on an element propagates upward through its ancestor elements in the DOM tree.

For example, when a button inside a div is clicked, the click event can first be handled by the button, then by the div, and then by other ancestors.

```
<div id="parent">
  <button id="child">Click Me</button>
</div>
<script>
  document.getElementById("parent").addEventListener("click", function() {
    console.log("Parent clicked");
  });
  document.getElementById("child").addEventListener("click", function() {
    console.log("Child clicked");
  });
</script>
```

O/P : Child clicked

Parent clicked

Explanation: The event handler on the button executes first. The event then bubbles upward to the parent div, causing its handler to execute.

Uses of event bubbling:

- Event delegation, where a parent handles events from multiple child elements.
- Reducing the number of individual event listeners.
- Handling events efficiently in dynamically generated elements.

7. What is the difference between named function, anonymous function, and arrow function?

Ans. JavaScript provides different ways to define functions. A named function has an explicit name, an anonymous function has no function name, and an arrow function uses the => syntax.

A. Named function

A named function is a function declared with a specific name. It can be called using that name.

Advantages: Easy to identify, reuse, and debug. Function declarations are also hoisted.

```
function addNumbers(firstNumber, secondNumber) {
  return firstNumber + secondNumber;
}
console.log(addNumbers(5, 3));
```

B. Anonymous function

An anonymous function is a function expression without its own explicit name. It is often assigned to a variable or passed as an argument to another function.

Advantages: Useful for callbacks and functions that are needed only in a particular context.

```
const addNumbers = function(firstNumber, secondNumber) {
  return firstNumber + secondNumber;
};
console.log(addNumbers(5, 3));
```

C. Arrow function

An arrow function is a shorter way to write a function using the => operator. It does not have its own this value; instead, it inherits this from its surrounding lexical scope.

For a single expression, an arrow function can be written more concisely:

```
const addNumbers = (firstNumber, secondNumber) => {
  return firstNumber + secondNumber;
};
console.log(addNumbers(5, 3));
```

Differences

Named function	**Anonymous function**	**Arrow function**
Has an explicit name.	Has no explicit function name.	Uses the => syntax.
Can be declared using function.	Commonly used as a function expression.	Commonly assigned to a variable or used as a callback.
Function declarations are hoisted.	Function expressions are not callable before initialization.	Arrow functions assigned to variables follow the variable's hoisting rules.
Has its own this depending on how it is called.	A regular function has its own this depending on how it is called.	Inherits this from its surrounding scope.
Can be used as a constructor when declared as a regular function.	A regular function expression can be used as a constructor.	Cannot be used as a constructor.

Conclusion: Named functions are useful for reusable operations, anonymous functions are useful for callbacks, and arrow functions provide concise syntax with lexical this behavior.

8.Explain for-of and for-in loop?

Ans. The for...of and for...in loops are used to iterate over data in JavaScript, but they serve different purposes.

A. for...of loop

The for...of loop iterates over the values of an iterable object, such as an array, string, or Set.

```
const numbers = [10, 20, 30, 40];
for (let number of numbers) {
  console.log(number);
}
```

B. for...in loop

The for...in loop iterates over the enumerable property keys of an object. When used with an ordinary array, it iterates over its enumerable index keys, which are usually strings.

```
Const student = {
  name: "Krishna",
  age: 20,
  course: "BTech"
};
for (let key in student) {
  console.log(key, student[key]);
}
```

Differences

for...of	**for...in**
Iterates over values.	Iterates over property keys.
Works with iterable objects.	Works with enumerable property keys of objects.
Suitable for arrays and strings.	Suitable for iterating over object properties.
Directly provides each array element.	Provides array index keys when used on ordinary arrays.

9. Explain any five array methods in JavaScript.

Ans. Array methods are built-in JavaScript functions that help perform operations such as adding elements, removing elements, transforming data, and filtering values.

The following are five commonly used array methods.

1. push()

The push() method adds one or more elements to the end of an array and returns the new length of the array.

2. pop()

The pop() method removes the last element from an array and returns the removed element.

3. map()

The map() method creates a new array by applying a callback function to every element of the original array. It does not modify the original array by itself.

4. filter()

The filter() method creates a new array containing only those elements that satisfy a specified condition.

5. reduce()

The reduce() method processes array elements using a callback function and combines them into a single accumulated result.

Method	**Purpose**	**Modifies the original array?**
push()	Adds elements to the end.	Yes
pop()	Removes the last element.	Yes
map()	Transforms each element into a new array.	No
filter()	Selects elements into a new array.	No
reduce()	Combines elements into a single result.	No, not by itself

10. What is a callback method?

A callback function is a function passed as an argument to another function so that it can be invoked by that function at an appropriate time.

Callbacks are commonly used for event handling, asynchronous operations, timers, and array methods such as map(), filter(), and reduce().

Explanation:

1. The greet() function accepts a name and a callback function.
2. It prints a greeting using the supplied name.
3. It invokes callback() to execute the function passed to it.
4. Therefore, showMessage() executes after the greeting is printed.

Types of callbacks

- Synchronous callback: Executes during the current function's execution, such as a callback passed to map().
- Asynchronous callback: Executes later, when an asynchronous operation or event reaches the appropriate point, such as a setTimeout() callback.

Advantages of callbacks:

- Allow functions to be reused with different behaviors.
- Help handle asynchronous operations.
- Make event-driven programming possible.

Disadvantages of callbacks:

- Deeply nested callbacks can make code difficult to read and maintain.
- Error handling can become complicated in complex asynchronous workflows.
- Nested asynchronous callbacks can lead to callback hell.